

Main Menu

MainMenu.

- Первая сцена, которая загружается сразу после экрана загрузки - Главное меню.

Снизу текст-примечание. Справа текущие рекорды игрока, которые сохраняются в Json файл и подгружаются из SaveManager'a при каждом включении данной сцены. Слева классическая панель: начать игру, настройки, выход.

Начать игру - вызывает `GameFlowManager.Instance.StartRun()`, который сам выбирает случайную карту через `LevelManager`.

Настройки - вызывают окошко настроек. Более подробно на странице `Settings System`.

Выход - закрывает данный application.

- На фоне пока что находиться серый экран. Как только игра будет достаточно функционировать - я сыграю один забег, запишу на видео и поставлю данный ролик на фон, затемнив картинку. Возможно сделаю небольшой паралакс эффект от движения мышью.



Главное меню (незаконченное).



SettingsController.

Buttons.

```

1 using UnityEngine;
2
3 public class StartGame : MonoBehaviour
4 {
5     public void StartGameClick()
6     {
7         if (SettingsMenuController.Instance.gameObject.activeInHierarchy)
8         {
9             SettingsMenuController.Instance.Hide();
10        }
11
12        GameFlowManager.Instance.StartRun();
13    }
14 }

```

```

1 using Unity.VisualScripting;
2 using UnityEngine;
3
4 public class Settings : MonoBehaviour
5 {
6     public void SettingsClick() =>
7         SettingsMenuController.Instance.Show();
8 }

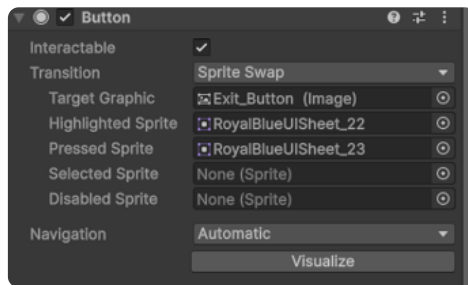
```

```

1 using UnityEngine;
2
3 public class ExitGame : MonoBehaviour
4 {
5     public void ExitButton()
6     {
7         FadeManager.FadeOutThen(() => {
8             #if UNITY_EDITOR
9                 UnityEditor.EditorApplication.isPlaying = false;
10            #else
11                Application.Quit();
12            #endif
13        }, 1.5f);
14 }

```

- ❗ У меня есть отдельные спрайты для кнопки при наведении или нажатии, однако этого мне показалось мало и я решил сделать кнопки более живыми и интересными.



В попытках что-то придумать, у меня получился эффект прилипания кнопок к курсору мыши.

```

1  using UnityEngine;
2  using UnityEngine.EventSystems;
3  #if ENABLE_INPUT_SYSTEM
4  using UnityEngine.InputSystem;
5  #endif
6
7  public class ButtonFollowMouse : MonoBehaviour, IPointerEnterHandler,
8  {
9      [Header("Settings")]
10     [SerializeField] public float moveAmount = 10f;
11     [SerializeField] public float smoothTime = 0.1f;
12     [SerializeField] public Camera camera;
13
14     private RectTransform rectTransform;
15     private Vector3 originalPosition;
16     private Vector3 velocity;
17
18     private bool isHovered = false;
19
20     private void Awake()
21     {
22         rectTransform = GetComponent<RectTransform>();
23         originalPosition = transform.localPosition;
24     }
25
26     // Запоминаем RectTransform и текущую позицию кнопки. Подключаем
27     // IPointerEnterHandler, ExitHandler и будем просто менять флаг, когда
28     // будет наводиться на объект, или уходить за границы объекта.
29
30     void Update()
31     {
32         if (isHovered)
33         {
34             Vector2 mousePos;
35             #if ENABLE_INPUT_SYSTEM
36                 mousePos = Mouse.current.position.ReadValue();
37             #else
38                 mousePos = Input.mousePosition;
39             #endif
40
41             // Vector2 mousePos - экранные координаты мыши в пикселях (например: 0
42             // Позиция кнопки на Canvas - локальные координаты внутри UI.
43             // Сравнивать их напрямую нельзя, кнопка будет вести себя не верно.

```

```

44
45         Vector2 localMousePos;
46         RectTransformUtility.ScreenPointToLocalPointInRectangle(
47             rectTransform,
48             mousePos,
49             camera,
50             out localMousePos
51         );
52
53     /*
54      * Метод RectTransformUtility.ScreenPointToLocalPointInRectangle берет
55      * экранную позицию мыши в пикселях, учитывает камеру, отслеживающую UI и
56      * размеры Canvas.
57      * После чего выдаёт позицию курсора относительно центра самой кнопки.
58      * (0, 0) = центр кнопки, (15, 0) = правее центра.
59      * Это запишется в localMousePos поле.
60      */
61
62         Vector2 clamped = Vector2.ClampMagnitude(localMousePos, mo
63
64     /*
65      * Теперь делаем ограничение движения кнопки через
66      * Vector2.ClampMagnitude(локальная позиция мыши, максимальный сдвиг).
67      * Clamp берёт вектор и проверяет его длину.
68      * Если мышка ушла далеко (условно на 100 пикселей), а moveAmount равен м
69      * то клэмп пропорционально сожмёт вектор до moveAmount.
70      * Как итог, мы получим вектор, куда надо двигаться, но не более того, чт
71      */
72         Vector3 targetPos = originalPosition + new Vector3(clamped
73
74     /*
75      * Теперь получим таргет - возьмём стандартную позицию кнопки, прибави
76      * (максимальный вектор.X, максимальный вектор.Y, 0).
77      * Например, позиция кнопки -500, 200. Если не прибавлять originalPositio
78      * то мы получим... Например -> (10, -5). В таком случае кнопка просто улет
79      * координаты. Поэтому в итоге мы получим (-490, 195), как координаты,
80      * куда кнопка должна подвинуться.
81      * Зачем конвертировать в Vector3? → rectTransform.localPosition мы бу
82      * двигать через SmoothDamp, который имеет Vector3.
83      */
84         rectTransform.localPosition = Vector3.SmoothDamp(rectTrans
85             targetPos, ref velocity, smoothTime);
86     }
87
88     /*
89      * В отличие от Mathf.Lerp, SmoothDamp симулирует реальную физическую
90      * или амортизатор, а не линейный или процентный шаг.
91      * Присваиваем позиции нашей кнопки SmoothDamp(текущая позиция, координат
92      * двигаться, велосити - информацию о скорости движения в предыдущем кадр
93      * (которая нужна для соблюдения правил энергии), время сглаживания
94      * (приблизительное время, за которое кнопка пройдёт большую часть пути))
95      */
96     else
97     {
98         rectTransform.localPosition = Vector3.SmoothDamp(rectTrans
99             originalPosition, ref velocity, smoothTime);
100     }
101
102     // Ну а если курсор мыши находится вне границ кнопки - мы аналогично с
103
104     public void OnPointerEnter(PointerEventData eventData)
105     {
106         isHovered = true;
107     }
108
109     public void OnPointerExit(PointerEventData eventData)
110     {

```

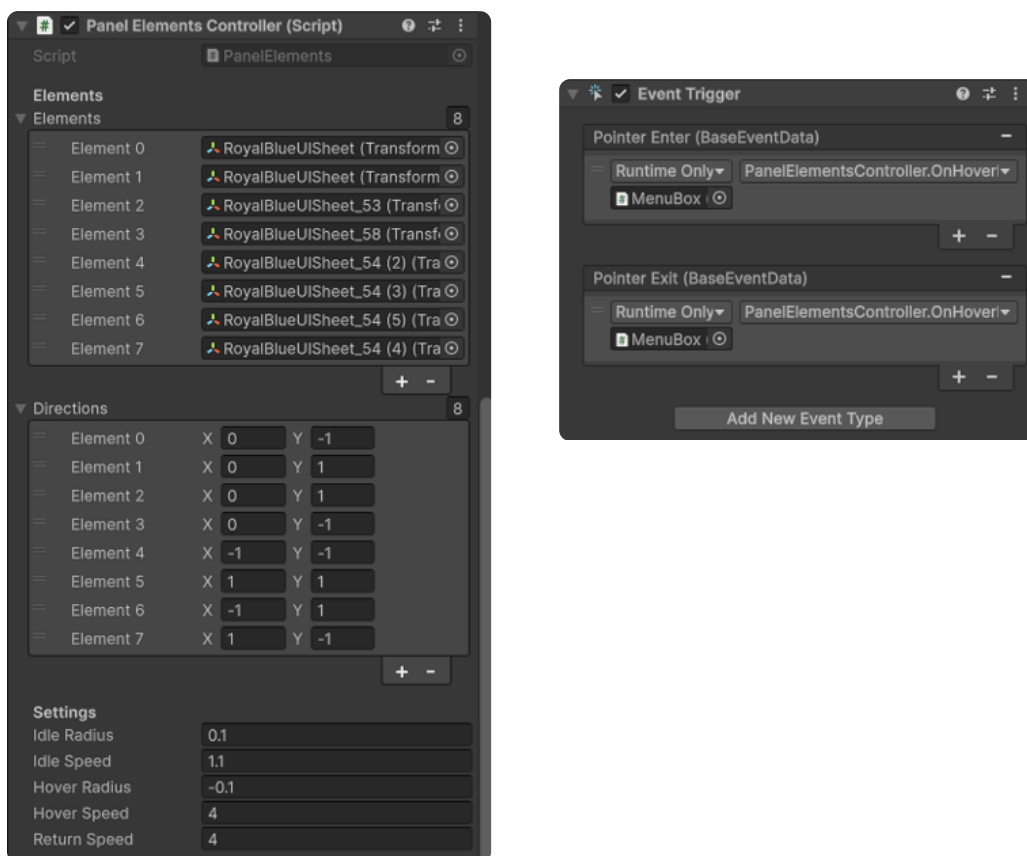
```

111         isHovered = false;
112     }
113 }

```

Animations Fragments.

- Чтобы было ещё более “живо”, было решено сделать анимацию остальных графических деталей. Идея в том, что все они двигаются в противоположном от меню направлении, а потом обратно. В случае наведения на блок меню, они прижимаются к нему и застывают.



Здесь isHover меняется иначе. Я для эксперимента повесил EventTriger, у которого встроены PointerEnter, PointerExit. И просто передал им ссылку на методы изменения isHover.

Придётся немного поработать ручками. Поэтому закидываем все элементы в нужном порядке, а потом для каждого элемента указываем его направление.

В массиве дирекшинс мы будем хранить направления: -1, 0, 1 для X, Y. Это будет вектор направления, в сторону которого могут двигаться элементы.

```

1 using UnityEngine;
2
3 public class PanelElementsController : MonoBehaviour
4 {
5     [Header("Elements")]
6     public Transform[] elements;
7     public Vector2[] directions;
8
9     [Header("Settings")]
10    public float idleRadius = 0.2f;
11    public float idleSpeed = 1.15f;
12    public float hoverRadius = -0.1f;
13    public float hoverSpeed = 4f;
14    public float returnSpeed = 4f;
15
16    private Vector3[] startPositions;
17    private bool isHover = false;
18    private float[] progress;
19    private int[] directionSign;
20
21    private Vector3[] returnPositions;
22
23    void Start()
24    {
25        if (elements == null || directions == null || elements.Length
26        {
27            Debug.LogError($"[PanelElementsController] Ошибка настройки на
28                $"Количество элементов ({elements?.Length}) не
29                $"Анимация полностью отключена.");
30
31            enabled = false;
32            return;
33        }
34
35        startPositions = new Vector3[elements.Length];
36        progress = new float[elements.Length];
37        directionSign = new int[elements.Length];
38        returnPositions = new Vector3[elements.Length];
39
40        for (int i = 0; i < elements.Length; i++)
41        {
42            startPositions[i] = elements[i].localPosition;
43            progress[i] = 0f;
44            directionSign[i] = 1;
45
46            if (directions.Length <= i)
47                directions = new Vector2[elements.Length];
48            directions[i] = directions[i].normalized;
49        }
50    }
51
52    /*
53     В Start пакуем стартовые позиции в массив, указываем прогресс как 0
54     (показывает насколько близко элемент к разрешенному радиусу idleRadius
55     directionSign - знак направления (-1, 1). В старте мы изначально всем
56     запуская маятник вперёд (пульсацию наружу).
57     */
58
59    void Update()
60    {
61        for (int i = 0; i < elements.Length; i++)
62        {
63            Vector3 targetPos;
64

```

```

65         if (isHover)
66         {
67             targetPos = startPositions[i] + new Vector3(directions
68                 elements[i].localPosition = Vector3.Lerp(elements[i].l
69         }
70         else
71         {
72             if ((elements[i].localPosition - (startPositions[i] +
73                 {
74                     targetPos = startPositions[i] + new Vector3(direct
75                     elements[i].localPosition = Vector3.Lerp(elements[
76                 }
77             else
78             {
79                 progress[i] += directionSign[i] * idleSpeed * Time
80
81                 if (progress[i] >= 1f)
82                 {
83                     progress[i] = 1f;
84                     directionSign[i] = -1;
85                 }
86                 else if (progress[i] <= 0f)
87                 {
88                     progress[i] = 0f;
89                     directionSign[i] = 1;
90                 }
91
92                 targetPos = startPositions[i] + new Vector3(direct
93                 elements[i].localPosition = targetPos;
94             }
95         }
96     }
97 }
98
99 public void OnHoverEnter()
100 {
101     isHover = true;
102 }
103
104 public void OnHoverExit()
105 {
106     isHover = false;
107 }
108 }
109
110 /*
111  В Update проверяем, если курсор наведён, то устанавливает таргет поз
112  для каждого элемента. В данном случае таргет позиция вычисляется так:
113  стартовая позиция + Вектор3(направлениеX, направлениеY, 0) умноженные н
114  (в нашем случае он минусовой, чтобы элементы сильнее прижались к
115  панели меню (внутрь)).
116  К примеру: верхний правый объект (синий угловой узор).
117  */

```

Его направление = (1, 1)

Получается правая верхняя

плоскость. Настоящая стартовая

позиция: (-3.1, 3.5).

Считаем таргет точку:
 $(-3.1, 3.5) + (1, 1, 0) * -0.1f = (-3, 3.4)$. Это точка в пространстве, что находится ближе к блоку меню, чем стартовая позиция объекта.

Далее сдвигаем через `Vector3.Lerp` (текущая позиция, куда сдвигаться, дельтаТайм * скорость) → линейная интерполяция.

Стартовое движение объекта будет достаточно быстрым, а вот по приближению к финальной точк элемент будет замедляться и плавно встанет на место.



Если же наведения мыши нету, то у элементов происходит плавная пульсирующая анимация.

Как только мышь уводится - наша задача плавно вернуть элементы на их места. В это время элемент находится очень далеко от своей траектории покачивания. В случае, если мы не укажем плавное возвращение, то элемент будет телепортироваться на нужную позицию.

Сначала сделаем проверку: Если (локальная позиция - (стартовая позиция + вектор направлений * радиус idle состояния * прогресс))) переведенная в магнитуду больше 0.01f) ...

Операция рассчитывает точку, где элемент должен был бы сейчас находиться в цикле покачивания.

Вычитает из текущей позиции кнопку эту целевую точку (А-Б).

А свойство `.magnitude` считает физическое расстояние в пикселях\единицах между этими двумя точками (длину вектора). Если расстояние больше 0.01f → значит элемент сбит с траектории покачивания.

Теперь плавно возвращаем его на нужную точку. Вычисляем таргет → стартовая позиция + вектор направления * радиус покачивания * прогресс.

После чего будем двигать через Lerp(позиция, таргет, дельтаТайм * скорость возвращения).

Как только элемент доплывает обратно и расстояние становится меньше 0.01f - управление переходит в ветку else, маятник покачивания подхватывает элемент и он снова плавно движется туда-сюда.

На старте прогресс = 0, дирекшинСайн = 1 (наружу). Перевычисляем прогресс: $0 += 1 * 1.15 * 0.016$ (в идеальных условиях). = 0,0184. Каждую секунду прогресс будет расти.

Проверяем, не стал ли прогресс равен 1ке - не достиг ли он конца разрешенной зоны. Если достиг → устанавливаем прогресс в 1, а дирекшинСайн (направление) устанавливаем на -1 (внутри).

Если прогресс достиг нуля (достиг границ внутреннего края) - то устанавливаем его в 0, а направление устанавливаем наружу.

Таргет = стартовая + вектор направления * доступный радиус * прогресс (направление (внутри, наружу), скорость, время).

Например: $(-3.1, 3.5) + 1,1 * 0.2 * 0.018 = (-3.10036, 3.50036)$.

Просто устанавливаем таргет как текущую позицию. Почему не плавный Lerp? - Разрыв между текущим положением элемента и вычисленным таргетом составляет менее 0.01 пикселя. Что крайне мало. Lerp бы наоборот сломал логику. Разница = 0.0036. Благодаря огромной скорости Update и низкой разнице позиций, перемещение всё равно выглядит очень плавным.

Забавно, что логика состоит во многом из направлений и просто смены плюса на минус у числа. Ведь если бы у нас были -1, -1 (движение вниз влево), то для данного элемента бы получили например -3.0964 по X, начав движение в обратную сторону. К слову, для противоположного (нижнего-левого) элемента -1, -1 будет двигать на старте его наоборот от панели. А потом направление сменится на 1, 1. Простая, но забавная логика 😊

⚠ Блок ниже также присутствует на странице Save System.

Records Panel и анимация бегущих чисел.



В скрипте панели рекордов, при включении объекта, мы получаем данные из менеджера и закидываем их на интерфейс.

```
1 using TMPro;
2 using UnityEngine;
3 using UnityEngine.UI;
4 public class RecordsPanel : MonoBehaviour
5 {
6     [SerializeField] public TextMeshProUGUI bestScoreText;
7     [SerializeField] public TextMeshProUGUI mostWavesText;
8     [SerializeField] public TextMeshProUGUI totalWavesText;
9     [SerializeField] public TextMeshProUGUI totalKilledText;
10    private void OnEnable()
11    {
12        var stats = SaveManager.Instance.Data.stats;
13        bestScoreText.text = stats.bestScore.ToString();
14        mostWavesText.text = stats.mostWaves.ToString();
15        totalWavesText.text = stats.totalWaves.ToString();
16        totalKilledText.text = stats.totalKilled.ToString();
17    }
18 }
```

i Статичный текст мне показался скучным и я думал что бы добавить... Мне пришло на ум сделать простенькую анимацию с переборкой цифр. Каждые пару секунд идёт переборка цифр в случайном порядке, а потом останавливается на нужном значении.

У меня будет бесконечный цикл, который будет крутить корутину, а после запускать задержку перед повторной анимацией.

```
1 using System.Collections;
2 using TMPro;
3 using UnityEngine;
4 using System.Text.RegularExpressions;
5 using System.Collections;
6 public class RecordsAnimator : MonoBehaviour
7 {
8     [Header("Fields")]
9     public TMP_Text[] recordFields;
10    [Header("Settings")]
11    public float initialDelay = 2.5f;
12    public float animationDuration = 2f;
13    public float repeatDelay = 7f;
14    public float updateInterval = 0.05f;
15
16    void Start()
17    {
18        StartCoroutine(AnimateRecordsLoop());
19    }
20
21    IEnumerator AnimateRecordsLoop()
22    {
23        yield return new WaitForSeconds(initialDelay);
24        while (true)
25        {
26            yield return StartCoroutine(AnimateRecords());
27            yield return new WaitForSeconds(repeatDelay);
28        }
29    }
```

```

30
31     IEnumerator AnimateRecords()
32     {
33         float timer = 0f;
34         int[] targetNumbers = new int[recordFields.Length];
35         // Сначала мы собираем все данные себе в массив.
36         for (int i = 0; i < recordFields.Length; i++)
37         {
38             string text = recordFields[i].text;
39             int num = 0;
40             Match match = Regex.Match(text, @"\d+");
41             /*
42              Попробуем из текста получить число. К слову, текст "MaxWave" -
43              у меня идёт отдельным объектом, и Regex можно было не использовать, но
44              @ - технический символ, чтобы работал обратный слеш.
45              \d - любая цифра (0-9).
46              + - одна или более подряд.
47             */
48             if (match.Success)
49             {
50                 num = int.Parse(match.Value);
51             }
52             targetNumbers[i] = num;
53         }
54         /*
55          То есть в определенный момент, если бы текст был цельным:
56          Максимум волн: 237, то метод пробежался бы по массиву символов,
57          дошёл до 2, и потом собирал бы цифры, пока снова не начались бы другие
58          В итоге мы получим 237. У меня цифры идут отдельным текстом, но как заш
59          предусмотреть можно. Далее мы парсим строку в инт тип.
60         */
61         while (timer < animationDuration)
62         {
63             timer += updateInterval;
64             for (int i = 0; i < recordFields.Length; i++)
65             {
66                 int digits = targetNumbers[i].ToString().Length;
67                 int randomValue;
68                 switch (digits)
69                 {
70                     case 1: randomValue = Random.Range(0, 10); break;
71                     case 2: randomValue = Random.Range(10, 100); break;
72                     case 3: randomValue = Random.Range(100, 1000); break;
73                     case 4: randomValue = Random.Range(1000, 10000); break;
74                     case 5: randomValue = Random.Range(10000, 100000); break;
75                     default: randomValue = targetNumbers[i]; break;
76                 }
77                 recordFields[i].text = randomValue.ToString();
78             }
79             yield return new WaitForSeconds(updateInterval);
80         }
81         /*
82          Запускаем сам цикл, длиной в некоторое время.
83          Через for присваиваем в каждое поле случайное число.
84          Но не абы какое, а равное по ширине тому, что изначально у нас находится
85          Если волн 237 - значит анимация будет от 100 - 999.
86          После чего мы просто выводим число на экран. И ожидаем некоторую задерж
87          Она кажется совсем маленькой = 0.05 секунды, однако без неё анимации бы
88          While бы просто срабатывал ультра быстро и выводил бы мгновенно прошлый
89          а так мы наблюдаем анимацию бегущих цифр.
90         */
91         for (int i = 0; i < recordFields.Length; i++)
92         {
93             recordFields[i].text = targetNumbers[i].ToString();
94         }
95         // В конце просто возвращаем на экран реальные значения.
96     }

```

